

Data Structures: Trees and Recursion

CMPT 211: Introduction to Software Development (Week 11)

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Transition: Beyond Linear Data Structures

Last week, we looked at **Stacks** and **Queues**.

While highly effective for tracking tasks or waiting lines, Stacks and Queues are **Linear Data Structures**. The data is arranged in a single, straight line (an array).

The Limitation of Linear Structures: If we have an array of 1,000,000 student records and we want to find one specific student, we might have to check every single item one by one. This is slow ($O(n)$ time complexity).

The Solution: Hierarchical Structures Today, we introduce **Trees**. By organizing data hierarchically (branching paths), we can cut our search time down exponentially.

The Binary Search Tree (BST) Concept

A **Binary Search Tree** is a dynamic data structure made of linked nodes.

The Three Rules of a BST:

- 1 Every node has a maximum of two children (**Left** and **Right**).
- 2 Every value in the **Left Subtree** must be strictly *less than* the parent's value.
- 3 Every value in the **Right Subtree** must be strictly *greater than* the parent's value.

Because of this strict mathematical ordering, every time we move down a branch, we eliminate half of the remaining data from our search! This is why it is called a "Search" tree.

Building the Foundation: The TreeNode Class

To build a tree, we must return to our Object-Oriented Programming (OOP) roots. We are not using flat arrays anymore. We are going to instantiate physical Node objects in memory and link them together.

```
class TreeNode:
    # The Constructor
    def __init__(self, key_value):
        # 1. The data payload (e.g., a student ID, a grade, or a simple integer)
        self.val = key_value

        # 2. The Pointers to the children.
        # They start as 'None' (Null) because a new leaf has no branches yet.
        self.left = None
        self.right = None
```

Every single circle in our Tree diagram will be an instance of this TreeNode blueprint.

Recursion: The Engine of Trees

Because a Tree is a self-similar structure (a branch of a tree looks exactly like a smaller tree), the most elegant way to navigate it is using **Recursion**.

What is Recursion? A recursive function is simply a function that calls itself from within its own code.

The Two Mandates of Recursion:

- 1 **The Base Case:** A strict condition that tells the function to STOP calling itself and return a value. Without this, you create an infinite loop and crash the program (Stack Overflow).
- 2 **The Recursive Step:** The logic that breaks the problem into a smaller piece and calls the function again.

Lab 11 Prep: The Recursive Insert Algorithm

In Lab 11, you must write a function to insert a new value into a BST while maintaining the strict left/right rules.

The Logic:

- Look at the current node.
- Is the spot empty? *Base Case: Insert here!*
- Is the new value smaller? *Recursive Step: Pass the value down to the Left child and repeat.*
- Is the new value larger? *Recursive Step: Pass the value down to the Right child and repeat.*

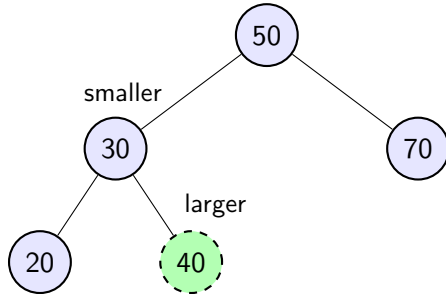
Translating Logic to Python (Lab 11 Solution Code)

Pay close attention to how we re-assign the `root.left` and `root.right` pointers to the results of the recursive calls. This is how the tree stitches itself together!

```
def insert(root, key):  
    # BASE CASE: We found an empty spot!  
    # Create the new Node object and return it up the chain.  
    if root is None:  
        return TreeNode(key)  
  
    # RECURSIVE STEP: Decide which branch to travel down.  
    if key < root.val:  
        # The value is smaller. Traverse LEFT.  
        # We catch the returned node and securely link it to our left pointer.  
        root.left = insert(root.left, key)  
  
    elif key > root.val:  
        # The value is larger. Traverse RIGHT.  
        root.right = insert(root.right, key)  
  
    # Finally, return the current, unmodified node back up to its parent  
    return root
```

Visualizing the Recursive Insert

Let's trace the execution of: `insert(root, 40)` into an existing tree.



The Trace:

- 1 Start at Root (**50**). 40 is smaller. Call `insert(root.left, 40)`.
- 2 Current node is (**30**). 40 is larger. Call `insert(root.right, 40)`.
- 3 Current node is **None** (Empty!). Base Case triggers. Create `TreeNode(40)`.
- 4 Return the new node up, linking it to the right pointer of **30**.

Questions?

Review Chapter 6 in Miller & Ranum.
Good luck with Lab 11 on LeetCode!